

Project on Hockey Analysis

By: Nikhitha Chenna

Table of Contents

1	Introduction	3
1.1	Setup Checklist for Project	3
1.2	Architecture	3
2	Problem Statement	4
2.1	Objective	4
2.2	Abstract of the project	4
2.3	Technology use	4
3	Implementation in PYSPARK	5
3.1	Summary of the functionality to be built	5
3.2	Guidelines on the functionality to be built	5
3.3	Implementation of project	6
4	Reports to be built	26
5	Appendix (source Files)	26

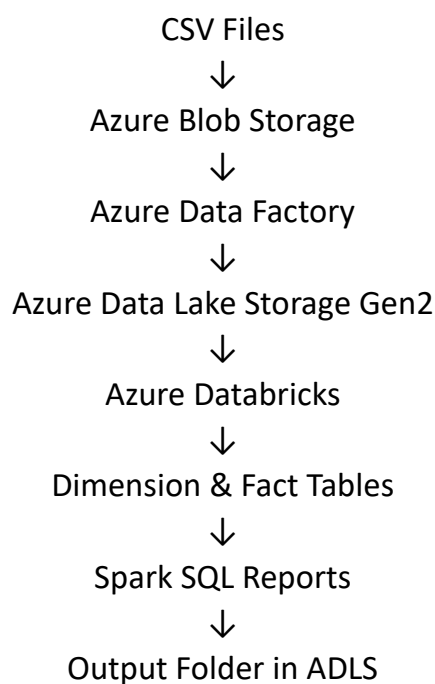
1 INTRODUCTION

This project demonstrates the implementation of an end-to-end Azure Data Engineering pipeline using Azure services and Databricks. The project focuses on processing hockey datasets, transforming raw data into fact and dimension tables, and generating analytical reports using PySpark and Spark SQL.

1.1 Setup Checklist for Project

- Azure Storage Account Created
- Azure Data Lake Storage Gen2 Enabled
- Azure Databricks Workspace Created
- Azure Data Factory Created
- Blob Container Configured
- CSV Files Uploaded
- Databricks Cluster Started
- ADLS Mounted in Databricks
- PySpark Environment Configured

1.2 Architecture of project



2 PROBLEM STATEMENT

2.1 Objective

To process hockey datasets stored in Azure Data Lake Storage using Azure Databricks and generate meaningful analytical reports using PySpark and Spark SQL.

2.2 Abstract of the Project

This project uses Azure Data Lake Storage Gen2, Azure Databricks, and PySpark to process football datasets. Raw CSV files are ingested into Databricks, transformed into dimension and fact tables, and used to generate analytical reports such as top goal scorers, richest clubs, and players with maximum appearances. The final reports are stored in the output folder of ADLS.

2.3 Technologies Used

- Azure Blob Storage
- Azure Data Lake Storage Gen2
- Azure Databricks
- Azure Data Factory
- PySpark
- Spark SQL
- Delta Tables

3 IMPLEMENTATION IN PYSPARK

3.1 Summary of Functionalities Built

- Mounted Azure Data Lake Storage in Databricks
- Read CSV files using PySpark
- Created Dimension Tables
- Created Fact Tables
- Performed ETL transformations
- Generated reports using Spark SQL
- Saved reports to ADLS output folder

3.2 Guidelines on Functionalities Built

- Step 1: Read Source Files
Loaded Hockey1.csv, Hockey2.csv, and date_dimension.csv into Databricks.
- Step 2: Create Dimension Tables
Created dim_player, dim_club, dim_country, dim_match, and dim_date tables.
- Step 3: Create Fact Tables
Created fact_player_stats and fact_team_stats tables using transformed datasets.
- Step 4: Create Reports
Generated analytical reports using Spark SQL queries.
- Step 5: Save Reports
Stored final report outputs into Azure Data Lake Storage output folder.

3.3 Implementation of project

Azure Blob Storage container was created successfully and the hockey dataset CSV files were uploaded into the rawdata container for further processing.

Microsoft Azure

Search resources, services, and docs (G+/I)



Home > Storage center | Blob Storage

Create a storage account

Instance details

Storage account name *

blobeeeee

Region *

(Asia Pacific) Central India

[Deploy to an Azure Extended Zone](#)

Preferred storage type

Azure Blob Storage or Azure Data Lake Storage

This helps us provide relevant guidance. It doesn't restrict your storage to this resource type. [Learn more](#)

Performance *

☒ Standard: Recommended for most scenarios (general-purpose v2 account)

☐ Premium: Recommended for scenarios that require low latency.

Redundancy *

Locally redundant storage (LRS)

Previous

Next

Review + create

Home > blobeeeee | Containers

rawdata

Container

Search

»

+ Add Directory

↑ Upload

🔒 Change access level

🔄 Refresh

🗑️ Delete

📄 Copy

📄 Paste

🏷️ Rename

🔑 Acquire lease

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

rawdata

Authentication method: Access key [\(Switch to Microsoft Entra user account\)](#)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 3 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease stat
☐	 Hockey1.csv	6/9/2026, 10:00:51 AM	Hot (Inferred)	Block blob	1.16 MiB	Available
☐	 Hockey2.csv	6/9/2026, 10:02:13 AM	Hot (Inferred)	Block blob	117.79 KiB	Available
☐	 date_dimension.c...	6/9/2026, 10:07:09 AM	Hot (Inferred)	Block blob	778.16 KiB	Available

Azure Data Lake Storage Gen2 was configured to store raw, processed, and transformed data generated during the ETL process.

Home > Storage center | Blob Storage

Create a storage account ...

Instance details

Storage account name * ⓘ

lakeeeee

Region * ⓘ

(Asia Pacific) Central India

Deploy to an Azure Extended Zone

Preferred storage type

Azure Blob Storage or Azure Data Lake Storage

ⓘ

This helps us provide relevant guidance. It doesn't restrict your storage to this resource type. [Learn more](#)

Performance * ⓘ

☒ Standard: Recommended for most scenarios (general-purpose v2 account)

☐ Premium: Recommended for scenarios that require low latency.

Redundancy * ⓘ

Locally redundant storage (LRS)

Home > Storage center | Blob Storage

Create a storage account ...

Basics

Advanced

Networking

Data protection

Security

Encryption

Tags

Review + create

Hierarchical Namespace

Hierarchical namespace, complemented by Data Lake Storage Gen2 endpoint, enables file and directory semantics, accelerates big data analytics workloads, and enables access control lists (ACLs) [Learn more](#) ⓘ

Enable hierarchical namespace ⓘ

☒

Access protocols

Blob and Data Lake Gen2 endpoints are provisioned by default [Learn more](#) ⓘ

Enable SFTP ⓘ

☐

Enable network file system v3 ⓘ

☐

Blob storage

Previous

Next

Review + create

Home > Storage center | Blob Storage > lakeeeee

lakeeeee | Containers

Storage account

Search

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

+ Add container

Upload

Refresh

Delete

Change access level

Restore containers

Search containers by prefix

Showing all 2 items

	Name	Last modified	Anonymous access level	Lease state
☐	\$logs	6/9/2026, 10:18:08 AM	Private	Available
☐	footballdata	6/9/2026, 10:20:18 AM	Private	Available

Success

Successful

pg. 7

Azure Databricks cluster was created and configured successfully to perform distributed Spark-based data transformations.

Home > Azure Databricks

Create an Azure Databricks workspace

Instance Details

Workspace name *	databrick
Region *	South India
Pricing Tier * ⓘ	Premium (+ Role-based access controls)

Workspace type

- ☐ Serverless
No setup required. Uses Databricks-managed default storage and serverless compute. Connect to your cloud storage at any time. [Learn more](#)
- ☒ Hybrid
Requires access to compute and storage in your Azure account. Supports custom compute.

[Review + create](#) [< Previous](#) [Next : Networking >](#)

Home

rg-azuser7172_mml.local-vZYWw_databrick | Overview

x << [Delete](#) [Cancel](#) [Redeploy](#) [Download](#) [Refresh](#)

Overview

- [Inputs](#)
- [Outputs](#)
- [Template](#)

✓ Your deployment is complete

 Deployment name : rg-azuser7172_mml.local-vZY... Start time : 6/9/2026, 10:39:29 AM
Subscription : [Learner Account](#) Correlation ID : bcf27244-3a2a-4a3a-acfb-0689...
Resource group : rg-azuser7172_mml.local-vZY...

> Deployment details

✓ Next steps

[Go to resource](#)

Microsoft Azure

databricks

Search data, notebooks, recents, and more... CTRL + P

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Compute > Simple form: ON

clusterrr

Send feed

Configuration Notebooks (0) Libraries Event log Spark UI Driver logs Metrics Apps Spark compute UI - Master

Summary

Data access

Price

16 GB Memory, 4 Cores

Unity Catalog

1 DBU/h

General

Compute name

clusterrr

Policy

Unrestricted

Performance

Machine learning

Databricks runtime

17.3 LTS (includes Apache Spark 4.0.0, Scala 2.13)

Photon acceleration

projecttt

Data factory (V2)

Show me integration runtime metrics for this Data factory (V2).

+2

Search

Delete

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Settings

Getting started

Monitoring

Automation

Help

Status

Succeeded

Location

Central India

Subscription (move)

Learner Account

Subscription ID

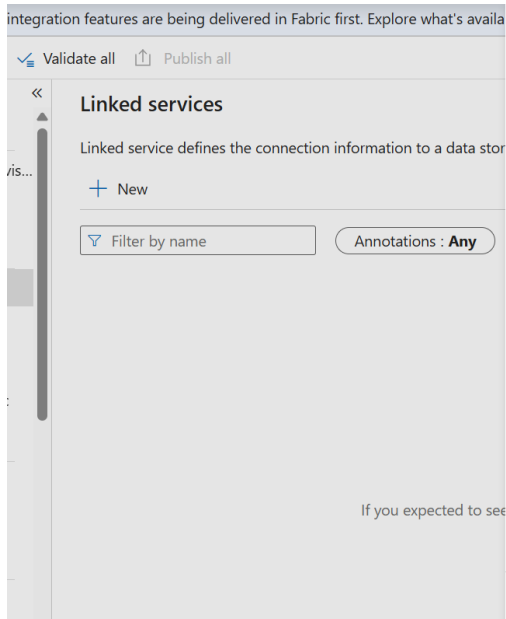
4116346b-5ded-4f3b-8387-f8d055802adc

Getting started

Quick start

Launch studio

Add or remove favorites by pressing Ctrl + Shift + F



New linked service

Azure Blob Storage [Learn more](#)

Authentication type

Account key

Connection string **Azure Key Vault**

Account selection method

☒ From Azure subscription ☐ Enter manually

Azure subscription

Learner Account (4116346b-5ded-4f3b-8387-f8d055802adc)

Storage account name *

blobeeeee

Additional connection properties

+ New

Test connection

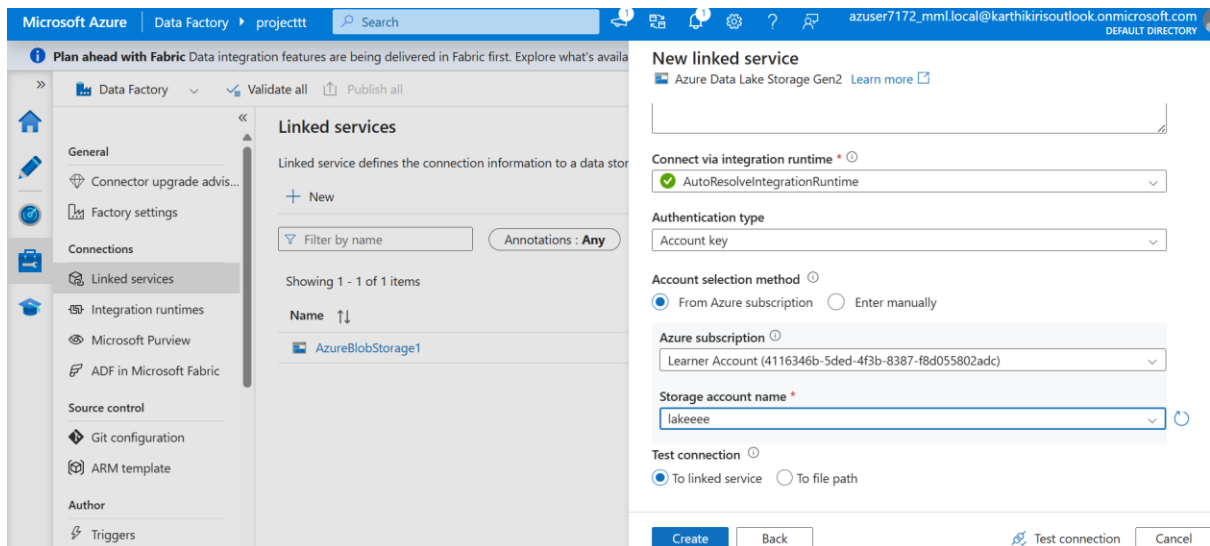
☒ To linked service ☐ To file path

Create

Back

Test connection

Cancel



Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#)

+ New

Filter by name

Annotations : Any

Showing 1 - 2 of 2 items

Name ↑↓	Type ↑↓	Related ↑↓	Annotations ↑↓
AzureBlobStorage1	Azure Blob Storage	1	
AzureDataLakeStorage1	Azure Data Lake Storage Gen2	1	

The screenshot displays the Azure Data Factory (ADF) interface. On the left, the 'Factory Resources' pane shows a tree view with 'Pipelines' (0), 'Change Data Capture (preview)' (0), 'Datasets' (2), 'sinkds', 'source', 'Data flows' (0), and 'Power Query' (0). The 'source' resource is selected, showing its configuration in the main pane. The 'Connection' tab is active, displaying 'Linked service *' as 'AzureBlobStorage1' and 'File path *' as 'rawdata'. Below this, there are links for 'Test connection', 'Edit', 'New', and 'Learn more'. The 'Sinkds' resource is also visible, showing a 'DelimitedText source' icon.

The second screenshot shows the 'Activities' pane with 'Copy data' selected. The 'Copy data1' activity is highlighted, and its 'Output' tab is active. The 'Output' tab shows a table with columns: 'Activity name', 'Activity st...', 'Activit...', 'Run start', and 'Duration'. The table contains one row: 'Copy data1', 'Succeeded', 'Copy data', '6/9/2026, 11:40:40 AM', and '15s'.

Azure Data Lake Storage was mounted successfully into Databricks using DBFS mount points for secure and efficient data access.

The screenshot shows the Databricks interface. At the top, a status bar indicates 'Just now (6s)' and '6' (likely the number of jobs or tasks). Below this, a code cell contains the command: `display(dbutils.fs.ls("/mnt/footballdata"))`. The output of the command is a table with 5 columns: 'path', 'name', 'size', and 'modificationTime'. The table contains 3 rows of data.

	path	name	size	modificationTime
1	dbfs:/mnt/footballdata/Hockey1.csv	Hockey1.csv	1218975	1780985452000
2	dbfs:/mnt/footballdata/Hockey2.csv	Hockey2.csv	120616	1780985452000
3	dbfs:/mnt/footballdata/date_dimension.c...	date_dimension.c...	796835	1780985452000

The hockey datasets were loaded into Spark DataFrames using PySpark for transformation and analytical processing.

Just now (5s) 7 Python

```
hockey1_df = spark.read.csv(
    "/mnt/footballdata/Hockey1.csv",
    header=True,
    inferSchema=True
)

display(hockey1_df)
```

(3) Spark Jobs

hockey1_df: pyspark.sql.dataframe.DataFrame = [Year: integer, Month: string ... 22 more fields]

Table +

	Year	Month	Match	Player Name	Club Name	country Name	Position
1	2010	January	International	Adrian	NA	Netherlands	LW
2	2010	February	International	Adrian	NA	Netherlands	LW
3	2010	March	International	Adrian	NA	Netherlands	LW

Just now (2s) 8 Python

```
hockey2_df = spark.read.csv(
    "/mnt/footballdata/Hockey2.csv",
    header=True,
    inferSchema=True
)

display(hockey2_df)
```

(3) Spark Jobs

hockey2_df: pyspark.sql.dataframe.DataFrame = [Year: integer, Match_Name: string ... 12 more fields]

Table +

	Year	Match_Name	League_Name	Club_Name	Coach	Country Name	Mana
1	2010	Club	LA Liga	Atheletico Madrid	NA	NA	Alvin Luca
2	2010	Club	LA Liga	Atheletico Madrid	NA	NA	Alvin Luca
3	2010	Club	LA Liga	Atheletico Madrid	NA	NA	Alvin Luca
4	2010	Club	LA Liga	Atheletico Madrid	NA	NA	Alvin Luca

Just now (2s) 9 Python

```
dates_df = spark.read.csv(
    "/mnt/footballdata/date_dimension.csv",
    header=True,
    inferSchema=True
)

display(dates_df)
```

(3) Spark Jobs

dates_df: pyspark.sql.dataframe.DataFrame = [date_key: integer, full_date: timestamp ... 22 more fields]

Table +

	date_key	full_date	day_of_week	day_num_in_month	day_num_overall	da
1	20080101	2008-01-01T00:00:00.000+00:00	2	1	1	Tuesd
2	20080102	2008-01-02T00:00:00.000+00:00	3	2	2	Wedne
3	20080103	2008-01-03T00:00:00.000+00:00	4	3	3	Thursc
4	20080104	2008-01-04T00:00:00.000+00:00	5	4	4	Fridav

Dimension tables were created to store descriptive information related to players, clubs, countries, matches, and dates. These tables help in organizing and analysing hockey data efficiently.

```
08:13 PM (4s) 10

dim_player = hockey1_df.select(
    col("Player Name").alias("player_name"),
    col("Position").alias("position"),
    col("Nationality").alias("nationality"),
    col("Jersey_No").alias("jersey_no"),
    col("D. O. B.").alias("dob")
).distinct()

display(dim_player)

(2) Spark Jobs

dim_player: pyspark.sql.dataframe.DataFrame = [player_name: string, position: string ... 3 more fields]
```

	player_name	position	nationality	jersey_no	dob
1	Danny Welbeck	ST	Portugal	88	24-Jul-82
2	Ferdinand	LB	Brasil	76	28-Sep-80
3	Jose Enrique	LB	Germany	34	22-Feb-83
4	Evans	LB	Brasil	56	6-Mar-82
5	David Villa	LW	England	63	11-Oct-83
6	Paul Scholes	CAM	Argentina	36	8-Nov-83
7	Lucas Piazon	LW	Netherlands	88	28-Jun-83
8	Samir Handanovic	GK	Netherlands	66	24-Jan-81

```
dim_club = hockey2_df.select(
    col("Club_Name").alias("club_name"),
    col("League_Name").alias("league_name"),
    col("Coach").alias("coach"),
    col("Manager").alias("manager"),
    col("Owner").alias("owner"),
    col("Country Name").alias("country_name")
).distinct()

display(dim_club)

(2) Spark Jobs

dim_club: pyspark.sql.dataframe.DataFrame = [club_name: string, league_name: string ... 4 more fields]
```

	club_name	league_name	coach	manager	owner	country_name
1	Schalke	Bundesliga	NA	Wanda Mayer	Adam Strickland	NA
2	Dortmund	Bundesliga	NA	Ursa Finch	Yasir Conley	NA
3	Arsenal	EPL	NA	Caesar Townsend	Brandon Townsend	NA
4	NA	NA	Lucas Hodges	NA	NA	Belgium
5	Bayern Leverkusen	Bundesliga	NA	Mollie Ferguson	Hilel Nixon	NA
6	Atheletico Bilbao	LA Liga	NA	Murphy Strong	Erich Bird	NA
7	Swanswa City	EPL	NA	Duncan Navarro	Caleb Cote	NA

```
dim_country = hockey1_df.select(
    col("country Name").alias("country_name")
).distinct()

display(dim_country)
```

▶ (2) Spark Jobs

>  dim_country: pyspark.sql.dataframe.DataFrame = [country_name: string]

Table ▾ +

	 country_name
1	Germany
2	NA
3	Argentina
4	Belgium
5	Italy
6	Spain
7	Mexico
8	Brasil
9	England

```
dim_match = hockey1_df.select(
    col("Match").alias("match_type")
).distinct()

display(dim_match)
```

▶ (2) Spark Jobs

>  dim_match: pyspark.sql.dataframe.DataFrame = [match_type: string]

Table ▾ +

	 match_type
1	International
2	Club

```
dim_date = dates_df.select(
    "date_key",
    "full_date",
    "day_name",
    "month_name",
    "fb_year",
    "quarter"
)

display(dim_date)
```

▶ (1) Spark Jobs

>  dim_date: pyspark.sql.dataframe.DataFrame = [date_key: integer, full_date: timestamp ... 4 more fields]

Table ▾ +

	 date_key	 full_date	 day_name	 month_name	 fb_year	 quarter	
1	20080101	2008-01-01T00:00:00.000+00:00	Tuesday	January	2008	1	
2	20080102	2008-01-02T00:00:00.000+00:00	Wednesday	January	2008	1	
3	20080103	2008-01-03T00:00:00.000+00:00	Thursday	January	2008	1	
4	20080104	2008-01-04T00:00:00.000+00:00	Friday	January	2008	1	
5	20080105	2008-01-05T00:00:00.000+00:00	Saturday	January	2008	1	

Fact tables were created to store transactional and measurable hockey statistics such as goals scored, appearances, fouls, cards, wins, and losses.

```
fact_player_stats = hockey1_df.select(
    col("Year").alias("year"),
    col("Month").alias("month"),
    col("Match").alias("match_type"),
    col("Player Name").alias("player_name"),
    col("Club Name").alias("club_name"),
    col("country Name").alias("country_name"),
    col("appearances"),
    col("goals scored").alias("goals_scored"),
    col("goals assist").alias("goals_assist"),
    col("total shots").alias("total_shots"),
    col("shots on target").alias("shots_on_target"),
    col("fouls made").alias("fouls_made"),
    col("fouls suffered").alias("fouls_suffered"),
    col("yellow card").alias("yellow_card"),
    col("red card").alias("red_card"),
    col("goals saved").alias("goals_saved"),
    col("goals conceded(stopped)").alias("goals_conceded"),
    col("total penalty").alias("total_penalty"),
    col("successful penalty").alias("successful_penalty"),
    col("salary")
)

display(fact_player_stats)
```

> fact_player_stats: pyspark.sql.dataframe.DataFrame = [year: integer, month: string ... 18 more fields]

	year	month	match_type	player_name	club_name	country_name	appearances
1	2010	January	International	Adrian	NA	Netherlands	
2	2010	February	International	Adrian	NA	Netherlands	
3	2010	March	International	Adrian	NA	Netherlands	
4	2010	April	International	Adrian	NA	Netherlands	
5	2010	May	International	Adrian	NA	Netherlands	
6	2010	June	International	Adrian	NA	Netherlands	
7	2010	July	International	Adrian	NA	Netherlands	
8	2010	August	International	Adrian	NA	Netherlands	
9	2010	September	International	Adrian	NA	Netherlands	
10	2010	October	International	Adrian	NA	Netherlands	

```
fact_team_stats = hockey2_df.select(
    col("Year").alias("year"),
    col("Match_Name").alias("match_name"),
    col("League_Name").alias("league_name"),
    col("Club_Name").alias("club_name"),
    col("Coach").alias("coach"),
    col("Manager").alias("manager"),
    col("appearances"),
    col("wins"),
    col("losts"),
    col("drawn"),
    col("clean sheets").alias("clean_sheets"),
    col("Net Worth").alias("net_worth")
)

display(fact_team_stats)
```

► (1) Spark Jobs

> fact_team_stats: pyspark.sql.dataframe.DataFrame = [year: integer, match_name: string ... 10 more fields]

	year	match_name	league_name	club_name	coach	manager	appearances
1	2010	Club	LA Liga	Athetico Madrid	NA	Alvin Lucas	
2	2010	Club	LA Liga	Athetico Madrid	NA	Alvin Lucas	

Delta tables were created successfully in Databricks to support optimized storage and analytical querying.

```
dim_player.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_player")

dim_club.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_club")

dim_country.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_country")

dim_match.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_match")

dim_date.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_date")
```



```
fact_player_stats.write.format("delta") \
  .mode("overwrite") \
  .option("overwriteSchema", "true") \
  .saveAsTable("fact_player_stats")

fact_team_stats.write.format("delta") \
  .mode("overwrite") \
  .option("overwriteSchema", "true") \
  .saveAsTable("fact_team_stats")
```

(2) Spark Jobs

```
fact_player_stats.createOrReplaceTempView("fact_player_stats")

fact_team_stats.createOrReplaceTempView("fact_team_stats")

dim_player.createOrReplaceTempView("dim_player")

dim_club.createOrReplaceTempView("dim_club")

dim_country.createOrReplaceTempView("dim_country")

dim_match.createOrReplaceTempView("dim_match")

dim_date.createOrReplaceTempView("dim_date")
```

Spark SQL query was used to generate the Top 10 Goal Scorers report by calculating the total goals scored by each player and sorting the results in descending order.

```
#REPORT 1 - Top 10 Goal Scorers
report1 = spark.sql("""

SELECT
  player_name,
  SUM(goals_scored) AS total_goals
FROM fact_player_stats
GROUP BY player_name
ORDER BY total_goals DESC
LIMIT 10

""")

display(report1)
```

▶ (2) Spark Jobs

> report1: pyspark.sql.dataframe.DataFrame = [player_name: string, total_goals: long]

Table ▾ +

	^A _C player_name	¹ ₃ total_goals
1	Gabriel	483
2	Eden Hazard	473
3	Tallulah Leach	471
4	Hernandez	468
5	Iker Casillas	465
6	Pedro	463
7	David De Gea	461
8	Mertesacker	461
9	Hugo	460
10	Ribery	460

This report identifies the player with the maximum number of international appearances based on the appearance's column from the dataset.

```
#Player With Maximum Appearances
report2 = spark.sql("""
SELECT
    player_name,
    SUM(appearances) AS total_appearances
FROM fact_player_stats
GROUP BY player_name
ORDER BY total_appearances DESC
LIMIT 1
""")

display(report2)
```

▶ (2) Spark Jobs

> report2: pyspark.sql.dataframe.DataFrame = [player_name: string, total_appearances: long]

Table ▾ +

	^A _C player_name	¹ ₃ total_appearances
1	Gerrad Pique	576

The Top 5 Successful Clubs report was generated successfully using Spark SQL by calculating the total number of wins achieved by each club and sorting the results in descending order.

```
#Top 5 Clubs With Maximum Wins
report3 = spark.sql("""
SELECT
    club_name,
    SUM(wins) AS total_wins
FROM fact_team_stats
GROUP BY club_name
ORDER BY total_wins DESC
LIMIT 5
""")
display(report3)
```

► (2) Spark Jobs

> report3: pyspark.sql.dataframe.DataFrame = [club_name: string, total_wins: long]

	club_name	total_wins
1	NA	1413
2	Liverpool	170
3	Sunderland	158
4	Bayern Munich	157
5	Man City	156

The Top 5 Players with Maximum Red Cards report was generated successfully using PySpark aggregation and sorting operations on the fact_player_stats table.

```
#Top 5 Players With Maximum Red Cards
report4 = fact_player_stats \
    .groupBy("player_name") \
    .agg(
        sum("red_card").alias("total_red_cards")
    ) \
    .orderBy(
        col("total_red_cards").desc()
    ) \
    .limit(5)

display(report4)
```

► (2) Spark Jobs

> report4: pyspark.sql.dataframe.DataFrame = [player_name: string, total_red_cards: long]

	player_name	total_red_cards
1	Barthez	89
2	Hulk	85
3	Mario Gomez	84
4	Daley Blind	84
5	Gabriel	83

The Top 5 Richest Clubs report was generated successfully by cleaning and transforming the net worth values using PySpark functions and calculating the total net worth of each club.

```
#Top 5 Richest Clubs
from pyspark.sql.functions import regexp_replace

report5 = fact_team_stats \
    .withColumn(
        "net_worth_clean",
        regexp_replace(
            regexp_replace(col("net_worth"), "\\$", ""),
            ",",
            ""
        ).cast("double")
    ) \
    .groupBy("club_name") \
    .agg(
        sum("net_worth_clean").alias("total_net_worth")
    ) \
    .orderBy(
        col("total_net_worth").desc()
    ) \
    .limit(5)

display(report5)
```

► (2) Spark Jobs

>  report5: pyspark.sql.dataframe.DataFrame = [club_name: string, total_net_worth: double]

Table ▼

+

	^A _C club_name	1.2 total_net_worth
1	NA	2482236828
2	Schalke	27707544
3	Sevilla FC	27671796
4	Man City	27658248
5	VFB Stuttgart	27119208

```
report1.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report1")
```

► (2) Spark Jobs

```
report2.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report2")
```

► (2) Spark Jobs

```
report3.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report3")

▶ (2) Spark Jobs

▶ 08:30 PM (23s) 27

report4.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report4")

▶ (2) Spark Jobs

▶ 08:30 PM (22s) 28 Python
```

footballdata > output

Authentication method: Access key (Switch to Microsoft Entra user account)

Showing all 5 items

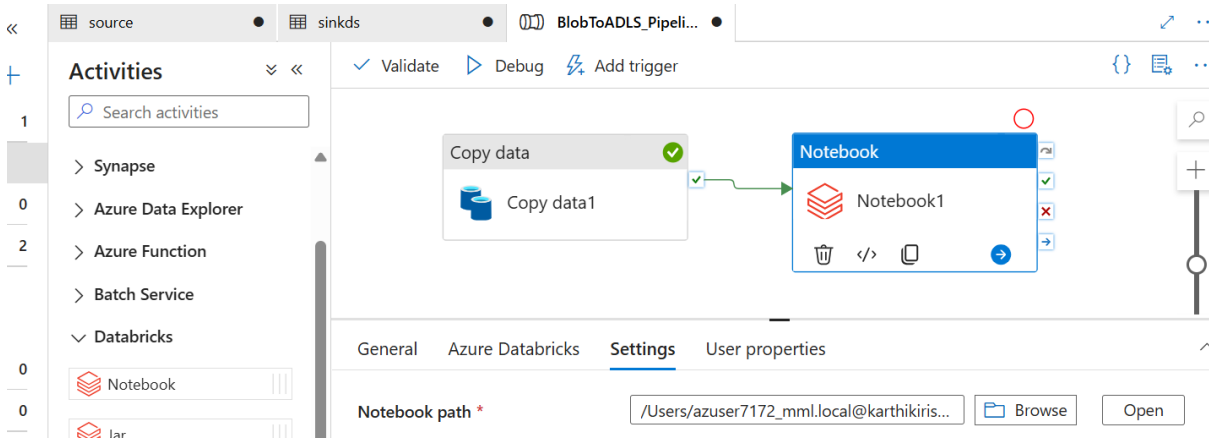
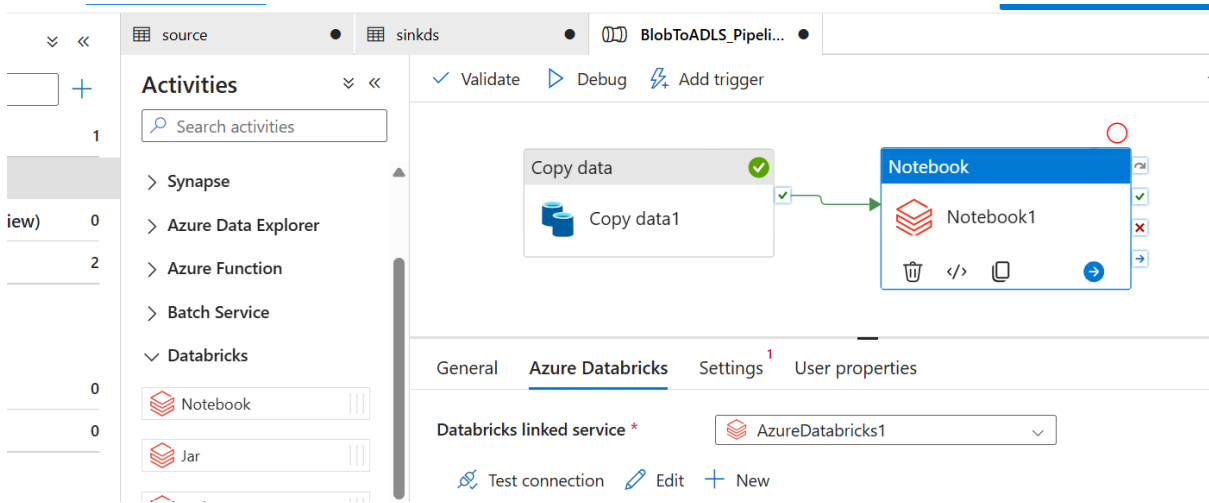
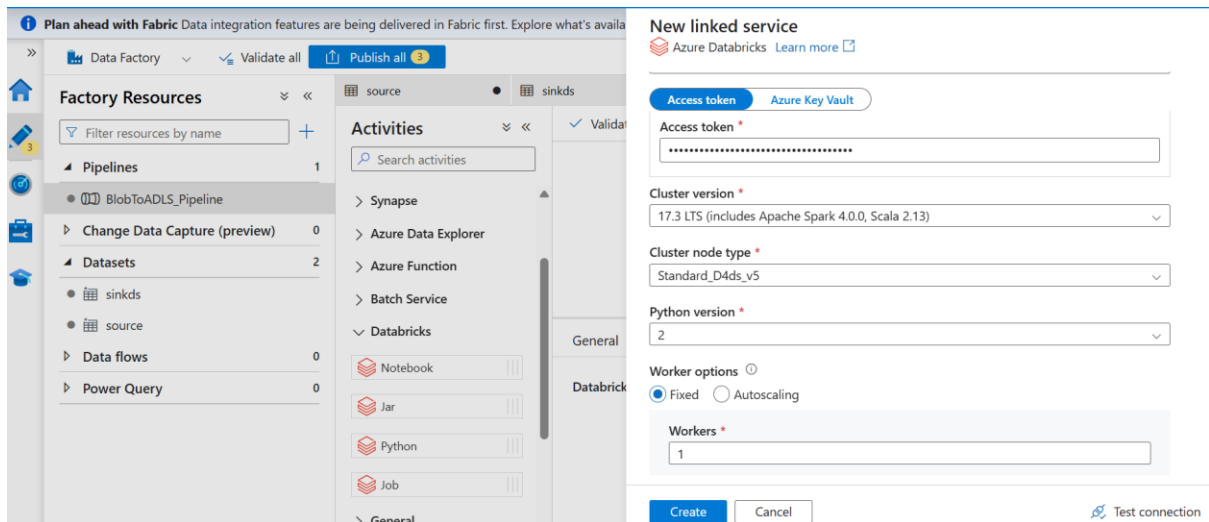
☐	Name	Last modified	Access tier	Blob type	Size
☐	[.]				
☐	report1	6/9/2026, 8:26:43 PM			
☐	report2	6/9/2026, 8:29:43 PM			
☐	report3	6/9/2026, 8:30:23 PM			
☐	report4	6/9/2026, 8:30:46 PM			
☐	report5	6/9/2026, 8:31:07 PM			

The Azure Data Factory pipeline executed successfully, completing data ingestion, notebook execution, Spark transformations, and report generation automatically.

This screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists various components: Pipelines (1), Datasets (2), sinkds, source, Data flows (0), and Power Query (0). The 'BlobToADLS_Pipeline' is selected. The main area displays the pipeline's execution flow, which includes a 'Copy data' activity followed by a 'Notebook' activity. The 'Output' tab shows the pipeline status as 'Succeeded' with a green checkmark. The pipeline ID is '4bbb1cd4-bd79-43b5-a467-372281f79015'.

This screenshot shows the 'New linked service' dialog in the Azure Data Factory console. The 'Azure Databricks' provider is selected. The 'From Azure subscription' option is chosen. The 'Azure subscription' dropdown shows 'Learner Account (4116346b-5ded-4f3b-8387-f8d055802adc)'. The 'Databricks workspace' dropdown shows 'databrick'. The 'Select cluster' dropdown shows 'New job cluster'. The 'Databricks Workspace URL' field contains 'https://adb-7405613391070005.5.azuredatabricks.net'. The 'Authentication type' dropdown shows 'Access Token'. The 'Access token' field is filled with a masked token. The 'Access token' and 'Azure Key Vault' buttons are visible.

This screenshot shows the Databricks interface. The 'Settings' pane is open, and the 'Generate new token' dialog is displayed. The dialog has fields for 'Name' (project), 'Lifetime (days)' (3), and 'Scope' (BI Tools, Other APIs). The 'API scope(s)' field shows 'all-apis'. The 'Generate' button is highlighted.



Data Factory | Validate all | Publish all | Migrate to Fabric (Preview)

Factory Resources

- Pipelines: 1
 - BlobToADLS_Pipeline
- Datasets: 2
 - Change Data Capture (preview)
- Sinks: 0
- Data flows: 0
- Power Query: 0

Activities

- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service
- Databricks
 - Notebook
 - Jar
 - Python
 - Job

Copy data → **Notebook**

Copy data1 → Notebook1

Parameters | Variables | Settings | **Output**

All status | Monitor in Azure Metrics | Export to CSV

Showing 1 - 2 of 2 items

Activity name	Activity st...	Activit...	Run start	Duration
Notebook1	✓ Succeeded	Notebook	6/9/2026, 2:02:52 PM	54s
Copy data1	✓ Succeeded	Copy data	6/9/2026, 2:02:36 PM	15s

The storage event trigger executed successfully and automatically started the pipeline whenever new CSV files were uploaded into Blob Storage.

source | sinkds | BlobToADLS_Pipeli...

Activities

- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service

Copy data → **Notebook**

Copy data1 → Notebook1

Trigger now

New/Edit

Plan ahead with Fabric Data integration features are being delivered in Fabric first. Explore what's available

Data Factory | Validate all | Publish all

Factory Resources

- Pipelines: 1
 - BlobToADLS_Pipeline
- Datasets: 2
 - Change Data Capture (preview)
- Sinks: 0
- Data flows: 0
- Power Query: 0

Activities

- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service
- Databricks
 - Notebook
 - Jar
 - Python
 - Job

New trigger

Azure subscription: Learner Account (4116346b-5ded-4f3b-8387-f8d055802adc)

Storage account name: blobeeeee

Container name: rawdata

Blob path begins with:

Blob path ends with:

Event: ☒ Blob created ☐ Blob deleted

Ignore empty blobs: ☒ Yes ☐ No

Continue Cancel

Plan ahead with Fabric Data integration | Validate all resources and publish them first. Explore what's available and plan your migration to Fabric. [Start exploring](#)

Data Factory | Validate all | Publish all

Factory Resources

- Pipelines: 1
 - BlobToADLS_Pipeline
- Datasets: 2
 - Change Data Capture (preview)
- Sinks: 0
- Data flows: 0
- Power Query: 0

Activities

- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service
- Databricks
 - Notebook
 - Jar
 - Python
 - Job

Trigger (1)

rawdata ...

Container

[+ Add Directory](#) [↑ Upload](#) [Change access level](#) [Refresh](#) [Delete](#) [Copy](#) [Paste](#) [Rename](#) [Acquire lease](#) ...

Overview

[Diagnose and solve problems](#)[Access Control \(IAM\)](#)[Settings](#)

rawdata

Authentication method: Access key ([Switch to Microsoft Entra user account](#))[Add filter](#) [Only show active blobs](#)

Showing all 4 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	 Hockey1.csv	6/9/2026, 10:00:51 AM	Hot (Inferred)	Block blob	1.16 MiB	Available
☐	 Hockey2.csv	6/9/2026, 10:02:13 AM	Hot (Inferred)	Block blob	117.79 KiB	Available
☐	 customers (1).csv	6/9/2026, 2:09:55 PM	Hot (Inferred)	Block blob	192 B	Available
☐	 date_dimension.c...	6/9/2026, 10:07:09 AM	Hot (Inferred)	Block blob	778.16 KiB	Available

footballdata ...

Container

[+ Add Directory](#) [↑ Upload](#) [Refresh](#) [Delete](#) [Copy](#) [Paste](#) [Rename](#) [Acqui](#)

Overview

[Diagnose and solve problems](#)[Access Control \(IAM\)](#)[Settings](#)

footballdata

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Showing all 6 items

☐	Name	Last modified	Access tier	Blob type
☐	 _\$azuretmpfolder\$	6/9/2026, 12:35:41 PM		
☐	 output	6/9/2026, 12:35:41 PM		
☐	 Hockey1.csv	6/9/2026, 2:10:16 PM	Hot (Inferred)	Block blob
☐	 Hockey2.csv	6/9/2026, 2:10:16 PM	Hot (Inferred)	Block blob
☐	 customers (1).csv	6/9/2026, 2:10:16 PM	Hot (Inferred)	Block blob

Runs

[Pipeline runs](#)[Trigger runs](#)[Change Data Capture \(previ...](#)

Runtimes & sessions

[Integration runtimes](#)[Data flow debug](#)

Filter by run ID or name	Chennai, Kolkata, Mu... : Last 24 hours	Pipeline name : All	Copy filters	Export to CSV	▼
Status : All	Runs : Latest runs	Triggered by : All	Add filter	×	
Showing 1 - 1 items					
☐	Pipeline name ↑↓	Run start ↑↓	Run end ↑↓	Duration	Triggered by
☐	BlobToADLS_Pipeline	6/9/2026, 2:10:00 PM	6/9/2026, 2:11:32 PM	1m 33s	trigger1
					✓ Succeed

4 REPORTS TO BE BUILT

1. Top 10 Goal Scorers

Generated a report of top goal scorers by calculating the total goals scored by each player.

2. Player with Maximum International Appearances

Identified the player with the maximum number of appearances using PySpark transformations.

3. Top 5 Successful Clubs

Generated a report identifying clubs with the maximum number of wins.

4. Top 5 Players with Maximum Red Cards

Generated a report identifying players with the highest number of red cards.

5. Top 5 Richest Clubs

Generated a report identifying clubs with the highest total net worth using PySpark transformations.

5 APPENDIX (SOURCE FILES)



Hockey Source
Data.zip

The following source files were used in the project:

- Hockey1.csv columns
- Hockey2.csv columns
- date_dimension.csv columns